

Pipeline Checkpoints (Priority Order)

P0: Core flow unblocks

- Complete `Agent.build_and_store()` in `src/services/signaling/agent.py`: train, save weights to Mongo, write metadata to Postgres.
- Align weight API names: use `get_agent_weights()` (or rename consistently) in `src/database_handlers/mongoDB.py` and `src/services/signaling/worker.py`.
- Ensure model input shape matches dataloader output (add projection or reshape in `src/services/signaling/dl.py` or update model in `src/services/signaling/model.py`).
- adjust the fetch dataloader to use the contextualizer
- replace the context vectors of finbert by finLang

P1: Training + launch lifecycle

- Implement Postgres metadata helpers (`create/update/get model`) in `src/database_handlers/postgres.py`.
- Wire worker launch path to read metadata, load weights, and start agent in `src/services/signaling/worker.py`.
- Ensure Redis tracking is consistent for active agents.

P2: API endpoints for orchestration

- Implement `start/stop/status/predict/list` endpoints in `src/server/flask_server.py` to drive the worker and inference RPC.
- Return a stable `model_id/agent_id` from `create_model()` so every downstream step can reference it.

P3: Live data integration

- Connect `fetch_inference_data()` to Refinitiv flow in `src/external_api/live.py` and ensure `.env` contains `REFINITIV_API_KEY` + credentials.
- Add retry/backoff handling for live data failures.

P4: Ops & readiness

- Health checks for RabbitMQ/Redis/Mongo/Postgres before starting worker/API.
- Minimal docs on `.env` required variables and example payloads.